



Security by design

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction : principes de security by design et privacy by design

Security by design

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définition

Security by design désigne l'approche consistant à intégrer les exigences de sécurité **dès la phase de conception** d'un système, plutôt que de les ajouter a posteriori sous forme de correctifs après un incident ou un audit. C'est un changement de posture : la sécurité devient une contrainte de conception au même titre que la performance ou l'ergonomie, pas une couche additionnelle optionnelle greffée en fin de projet.

Concrètement, cela signifie que les questions de sécurité doivent être posées dès les premières réunions de cadrage d'un projet : quelles données seront traitées, qui doit pouvoir y accéder, quelles menaces pèsent sur l'architecture envisagée, quelles conséquences en cas de défaillance. Ces questions influencent directement les choix structurants (découpage en composants, modèle de données, choix technologiques), et non uniquement des détails d'implémentation traités « plus tard ». Un système conçu ainsi porte la sécurité dans son architecture même, ce qui la rend beaucoup plus difficile à contourner qu'un contrôle ajouté en périphérie d'un système déjà figé.

À l'opposé, une approche « sécurité a posteriori » traite la sécurité comme une liste de vérifications à cocher juste avant la mise en production, ou pire, comme une réaction à un incident déjà survenu. Ce mode de fonctionnement produit typiquement des correctifs partiels, des contournements fragiles et une dette de sécurité qui s'accumule au fil des versions successives du logiciel.

2. Pourquoi corriger après coup coûte plus cher

Le coût de correction d'une faille de conception augmente très fortement à mesure que le projet avance dans son cycle de vie. Une faille identifiée en phase de conception se corrige souvent par un simple ajustement de schéma ou de choix d'architecture, sans code déjà écrit à modifier. La même faille découverte en production peut, elle, nécessiter de repenser une architecture déjà déployée, d'interrompre un service utilisé par des clients réels, de migrer des données existantes et de communiquer sur un incident — autant de coûts qui ne se limitent pas au seul temps de développement.

Phase de détection	Nature de la correction	Ampleur typique de l'effort
Exigences / conception	Ajustement d'un modèle de données ou d'une architecture pas encore implémentée	Faible : quelques heures de discussion et de re-conception
Développement	Modification de code non encore livré, avant intégration	Modérée : réécriture localisée, tests à adapter
Tests / recette	Correctif sur du code déjà intégré, potentiellement avec effets de bord sur d'autres fonctionnalités	Plus élevée : régression à vérifier sur l'ensemble du périmètre testé
Production	Correctif d'urgence sur un système en service, migration de données, communication aux utilisateurs, gestion d'un incident éventuel	Élevée : indisponibilité possible, risque de perte de confiance, coût organisationnel en plus du coût technique

Cette observation — qu'un ordre de grandeur sépare typiquement le coût d'une correction précoce de celui d'une correction tardive — motive l'intégration précoce de la sécurité, résumée par l'expression « **shift left** » : décaler les activités de sécurité vers la gauche d'une frise chronologique du projet, c'est-à-dire vers les phases amont. Ce principe est développé plus en détail au chapitre 5 consacré au DevSecOps, où il se traduit en pratiques concrètes intégrées au pipeline de développement.

3. Security by design vs sécurité périmétrique

Un modèle de sécurité historique reposait sur une **frontière protégée** (pare-feu, périmètre réseau) séparant un extérieur considéré hostile d'un intérieur considéré de confiance : une fois à l'intérieur du périmètre, un utilisateur ou une machine bénéficiait d'une confiance largement implicite, avec peu de contrôles supplémentaires. Ce modèle a longtemps constitué l'approche par défaut de la sécurité des systèmes d'information, en particulier lorsque l'essentiel des ressources informatiques restait physiquement localisé dans des locaux contrôlés par l'organisation.

Ce modèle s'avère cependant fragile dès qu'un attaquant parvient à franchir le périmètre — par hameçonnage d'un employé, compromission d'un poste de travail, ou via un prestataire tiers disposant d'accès internes légitimes mais mal maîtrisés. Une fois à l'intérieur, l'attaquant profite précisément de cette confiance implicite pour se déplacer latéralement d'un système à l'autre, souvent sans rencontrer de résistance significative avant d'atteindre sa cible réelle.

Modèle périmétrique	Security by design
Confiance forte accordée à tout ce qui est « à l'intérieur » du réseau	Aucune confiance implicite, quelle que soit l'origine de la requête
Contrôles concentrés à la frontière (pare-feu, VPN d'entrée)	Contrôles répartis à chaque composant et à chaque accès
Une compromission initiale ouvre un large accès interne	Une compromission partielle reste contenue par conception
Hypothèse implicite : le réseau interne est sûr	Hypothèse explicite : la compromission est possible, il faut la limiter

Security by design pousse ainsi à concevoir des systèmes résilients même en présence d'une compromission partielle, plutôt que de miser uniquement sur l'inviolabilité d'une frontière. Cette logique préfigure directement le modèle **zero trust**, développé au chapitre 4, qui généralise ce principe à l'ensemble de l'architecture réseau et applicative.

4. Les grands principes, aperçu

Concevoir un système sécurisé ne repose pas sur une intuition isolée mais sur un ensemble de principes de conception éprouvés, applicables indépendamment de la technologie utilisée. Ce module en détaille une liste au chapitre 3 : moindre privilège, défense en profondeur, échec sécurisé, séparation des tâches, simplicité de conception et médiation complète en constituent le noyau. Ces principes ont été formalisés dès 1975 par Jerome Saltzer et Michael Schroeder dans un article fondateur sur la protection des systèmes informatiques, et restent aujourd'hui d'une actualité remarquable malgré la transformation radicale des technologies concernées depuis cette date.

L'intérêt de ces principes réside précisément dans leur généralité : ils s'appliquent aussi bien à la conception d'un système d'exploitation qu'à celle d'une API web moderne ou d'une architecture Cloud distribuée. Le chapitre 3 les présente en détail, avec des exemples concrets d'application dans des contextes contemporains de développement logiciel.

5. Privacy by design

Concept complémentaire à la sécurité au sens strict, le *privacy by design* (protection de la vie privée dès la conception) a été formalisé par Ann Cavoukian, alors commissaire à l'information et à la protection de la vie privée de l'Ontario, à la fin des années 1990 et au cours des années 2000. Son principe central est que la protection des données personnelles doit être une **caractéristique par défaut** d'un système — présente sans action requise de l'utilisateur — plutôt qu'une option qu'il faudrait activer explicitement pour en bénéficier.

Ce concept, longtemps considéré comme une bonne pratique parmi d'autres, est aujourd'hui explicitement exigé par plusieurs réglementations de protection des données, au premier rang desquelles l'article 25 du RGPD (Règlement Général sur la Protection des Données de l'Union européenne), intitulé « protection des données dès la conception et par défaut ». Cette exigence juridique transforme une bonne pratique en obligation formelle pour tout responsable de traitement de données personnelles. Le chapitre 6 de ce module développe ce concept en détail, avec ses sept principes fondateurs et leurs implications pratiques.

6. Sécurité et fonctionnalité : un arbitrage, pas une opposition

Une idée reçue largement répandue oppose systématiquement sécurité et facilité d'usage, comme si l'une ne pouvait progresser qu'au détriment de l'autre. Cette vision binaire est trompeuse : une conception mature cherche plutôt à rendre le **comportement sécurisé le comportement par défaut et le plus simple** pour l'utilisateur, de sorte que suivre le chemin le plus naturel soit aussi le chemin le plus sûr.

Un exemple classique illustre ce principe : l'authentification à double facteur (MFA) apporte une réduction significative du risque de compromission de compte, mais son efficacité réelle dépend entièrement de son taux d'adoption. Si elle est proposée comme un réglage avancé, enfoui dans un menu que peu d'utilisateurs découvrent spontanément, son taux d'adoption reste faible et sa valeur protectrice réelle s'en trouve fortement amoindrie. Si elle est au contraire **activée par défaut**, avec un parcours d'activation simple, elle protège la totalité de la base d'utilisateurs sans effort supplémentaire de leur part.

À l'inverse, une mesure de sécurité qui dégrade excessivement l'expérience utilisateur est très souvent contournée en pratique, quelle que soit sa robustesse théorique : des mots de passe trop complexes et changés trop fréquemment finissent notés sur un post-it ou dans un fichier texte non protégé ; un contrôle jugé trop contraignant par les utilisateurs finit désactivé dès que la procédure le permet, ou évité par des chemins de contournement informels. Ce constat n'est pas un problème de discipline des utilisateurs qu'il faudrait corriger par la formation seule ; c'est avant tout un **échec de conception**, qui doit être corrigé en repensant l'ergonomie du contrôle de sécurité lui-même.

7. Articulation avec le reste du programme

Ce module de *security by design* ne fonctionne pas en vase clos : il synthétise et opérationnalise des notions abordées ailleurs dans le programme de formation. Le module *Audit organisation et technique* fournit le catalogue de vulnérabilités concrètes que la conception cherche justement à prévenir en amont, plutôt que de les découvrir a posteriori lors d'un audit. Le module *Gouvernance* apporte la méthodologie de gestion des risques permettant de prioriser les efforts de conception sécurisée en fonction de leur impact réel. Le module *Cryptographie* fournit les primitives techniques (chiffrement, signatures, fonctions de hachage) que la conception doit savoir intégrer correctement dans une architecture, sans en réinventer les mécanismes internes.

Security by design est ainsi la discipline qui transforme ces connaissances théoriques et ces catalogues de risques en pratiques de conception concrètes, appliquées en amont du développement plutôt qu'en réaction à un problème déjà survenu. C'est le fil conducteur qui relie l'analyse de risque, la cryptographie et l'audit à des décisions d'architecture effectives.

À retenir

- *Security by design* intègre la sécurité dès la conception, réduisant fortement le coût de correction par rapport à une approche corrective a posteriori — un ordre de grandeur sépare typiquement une correction en conception d'une correction en production.
- Le modèle de sécurité périmétrique seul est insuffisant face à des attaquants capables de franchir la frontière ; une conception résiliente à la compromission partielle est nécessaire, préfigurant le modèle zero trust (chapitre 4).
- Les grands principes de conception sécurisée (Saltzer et Schroeder, 1975) restent le socle théorique du domaine et sont détaillés au chapitre 3 ; le privacy by design, complément centré sur les données personnelles, est développé au chapitre 6.
- Une mesure de sécurité qui dégrade excessivement l'usage est souvent contournée : la sécurité par défaut et simple d'usage est un objectif de conception, pas un vœu pieux.
- Ce module opérationnalise des notions vues dans les modules Audit, Gouvernance et Cryptographie, en les traduisant en pratiques de conception concrètes.

Chapitre 2 — Modélisation des menaces (threat modeling)

Security by design

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Qu'est-ce que le threat modeling ?

Le threat modeling est une démarche structurée visant à identifier, de façon systématique et le plus en amont possible, les menaces plausibles pesant sur un système, afin de concevoir des mesures de sécurité proportionnées avant même l'implémentation. C'est l'un des outils centraux de *security by design* : là où un test d'intrusion cherche des vulnérabilités dans un système déjà construit, le threat modeling cherche à anticiper les faiblesses potentielles pendant que le système est encore sur le papier, quand les corriger coûte le moins cher (chapitre 1).

L'objectif n'est pas de produire une liste exhaustive et théorique de toutes les attaques imaginables — un exercice qui deviendrait vite ingérable et peu actionnable — mais de raisonner de façon structurée sur les menaces les plus pertinentes pour le système concret étudié, afin d'en tirer des décisions de conception concrètes. Un threat model n'est donc pas un document figé produit une fois pour toutes : c'est un outil de travail vivant, révisé à mesure que le système évolue.

2. La démarche générale en quatre questions (Adam Shostack)

Adam Shostack, référence reconnue du domaine, résume la démarche de threat modeling en quatre questions simples mais structurantes, à se poser dans cet ordre :

1. **Sur quoi travaillons-nous ?** Il s'agit de modéliser le système avant de chercher des menaces : produire un diagramme de flux de données, identifier les composants, les magasins de données et les frontières de confiance. Sans cette étape préalable, l'identification des menaces devient impressionniste et incomplète.
2. **Qu'est-ce qui peut mal tourner ?** C'est l'étape d'identification systématique des menaces proprement dite, appuyée sur des méthodes structurantes comme STRIDE (section 4) pour éviter les oublis.
3. **Que faisons-nous à ce sujet ?** Pour chaque menace retenue comme pertinente, il faut définir une contre-mesure concrète : un contrôle technique, un changement d'architecture, ou une décision explicite d'accepter le risque si sa probabilité et son impact le justifient.
4. **Avons-nous fait du bon travail ?** La démarche se termine par une validation : le threat model a-t-il été mené avec la rigueur nécessaire, les contre-mesures identifiées ont-elles réellement été mises en œuvre, et la démarche doit-elle être répétée à la prochaine évolution significative du système ?

Cette structuration en quatre questions a l'avantage d'être facilement mémorisable et transposable à des contextes très variés, du threat modeling formel en atelier à une réflexion plus légère menée par une petite équipe de développement.

3. Diagramme de flux de données (DFD) et frontières de confiance

Avant d'identifier des menaces, il faut représenter le système de façon suffisamment précise pour raisonner dessus. Un **diagramme de flux de données (DFD)** représente typiquement quatre types d'éléments : les **processus** (composants qui traitent des données, ex. un service applicatif), les **flux de données** (échanges entre composants), les **magasins de données** (bases de données, fichiers, caches) et les **entités externes** (utilisateurs, systèmes tiers avec lesquels le système interagit).

L'élément le plus important de ce diagramme, du point de vue de la sécurité, est la **frontière de confiance** (*trust boundary*) : un point du diagramme où le niveau de confiance accordé aux données ou à l'entité change. Par exemple, la frontière entre un client non authentifié sur Internet et un serveur applicatif interne, ou la frontière entre une zone démilitarisée (DMZ) exposée publiquement et un réseau interne considéré plus sensible.

Les menaces se concentrent typiquement sur ces frontières, car c'est précisément à ces points que des données ou des requêtes passent d'un niveau de confiance à un autre sans garantie automatique de leur légitimité. Un threat modeling efficace porte donc une attention particulière à chaque frontière représentée sur le diagramme, bien plus qu'au contenu interne d'un composant qui ne franchit aucune frontière.

4. STRIDE : classification des menaces

STRIDE est un modèle de classification des menaces développé par Microsoft, structuré en six catégories à examiner systématiquement pour chaque composant et chaque flux du diagramme de flux de données. Son intérêt principal est de servir d'aide-mémoire structurée, garantissant qu'aucune grande catégorie de menace n'est oubliée lors de la revue d'un composant.

Lettre	Menace	Propriété de sécurité visée	Exemple
S	Spoofing (usurpation d'identité)	Authenticité	Se faire passer pour un autre utilisateur, par exemple en réutilisant un jeton de session volé
T	Tampering (falsification)	Intégrité	Modifier des données en transit (interception d'une requête non protégée) ou au repos (modification directe d'un enregistrement en base)
R	Repudiation (répudiation)	Non-répudiation	Nier avoir effectué une action, en l'absence de journalisation suffisamment fiable pour la démontrer a posteriori
I	Information disclosure (divulgaration d'information)	Confidentialité	Accéder à des données sans autorisation, par exemple via un message d'erreur trop verbeux révélant des détails internes
D	Denial of Service (déni de service)	Disponibilité	Rendre un service indisponible, par exemple en saturant volontairement une ressource limitée (bande passante, connexions, calcul)
E	Elevation of Privilege (élévation de privilège)	Autorisation	Obtenir des droits supérieurs à ceux normalement accordés, par exemple en exploitant une vérification d'autorisation incomplète

Pour chaque flux et chaque composant du diagramme, la méthode consiste à se poser systématiquement la question : « ce composant est-il vulnérable à S ? à T ? à R ? » et ainsi de suite pour les six lettres. Un magasin de données, par exemple, est rarement concerné par le spoofing en tant que tel, mais reste très concerné par la falsification et la divulgation d'information — l'exercice aide justement à raisonner composant par composant plutôt que de façon globale et vague.

5. DREAD : évaluation de la sévérité

Une fois des menaces identifiées via STRIDE, toutes ne présentent pas le même niveau de risque : il devient nécessaire de les prioriser pour concentrer l'effort de correction sur les plus critiques. **DREAD** est un modèle complémentaire, historiquement associé à STRIDE, permettant de scorer une menace identifiée selon cinq critères indépendants :

- **Damage** (dommage potentiel) : quelle serait l'ampleur des dégâts si la menace se concrétisait ?
- **Reproducibility** (reproductibilité) : l'attaque est-elle facile à reproduire de façon fiable ?
- **Exploitability** (facilité d'exploitation) : quel niveau de compétence ou d'outillage est requis pour l'exploiter ?
- **Affected users** (utilisateurs affectés) : quelle proportion des utilisateurs serait touchée ?
- **Discoverability** (facilité de découverte) : la vulnérabilité est-elle facile à découvrir par un attaquant, y compris un attaquant peu qualifié ?

DREAD est aujourd'hui moins utilisé de façon isolée qu'à ses débuts, en partie à cause de la subjectivité de certains critères d'un évaluateur à l'autre ; il est souvent remplacé en pratique par une matrice de risque classique (probabilité × impact, cf. module *Gouvernance*) ou par le score **CVSS** (Common Vulnerability Scoring System) lorsque la menace correspond à une vulnérabilité déjà cataloguée et publiquement documentée.

6. Arbres d'attaque (attack trees)

Un **arbre d'attaque** offre une représentation hiérarchique et visuelle des chemins qu'un attaquant pourrait emprunter pour atteindre un objectif donné. Le nœud racine de l'arbre représente l'objectif final de l'attaquant (par exemple « compromettre le compte administrateur »), décomposé en sous-objectifs représentés par des nœuds enfants, reliés entre eux par des relations logiques **ET** (tous les sous-objectifs doivent être atteints) ou **OU** (un seul suffit), jusqu'à atteindre des actions concrètes et réalisables au niveau des feuilles de l'arbre.

Cette structure présente un double intérêt pédagogique et opérationnel. D'une part, elle aide à visualiser l'ensemble des chemins d'attaque possibles vers un objectif donné, plutôt que de se limiter au premier scénario venu à l'esprit. D'autre part, elle permet d'identifier des points de contrôle stratégiques : une contre-mesure placée à un nœud partagé par plusieurs branches de l'arbre peut bloquer simultanément plusieurs chemins d'attaque distincts, ce qui en fait un investissement de sécurité particulièrement rentable.

7. PASTA et autres méthodologies

Au-delà de STRIDE, d'autres méthodologies de threat modeling existent, avec des orientations différentes selon le contexte et les priorités de l'organisation :

Méthodologie	Orientation	Cas d'usage typique
STRIDE	Technique, par catégorie de propriété de sécurité	Revue systématique d'une architecture ou d'un composant
PASTA (Process for Attack Simulation and Threat Analysis)	Centrée sur le risque métier, simule des scénarios d'attaque complets de bout en bout	Contextes où l'alignement avec les enjeux métier est prioritaire
LINDDUN	Spécifiquement dédiée aux menaces sur la vie privée	Systèmes traitant des données personnelles sensibles (cf. chapitre 6, <i>privacy by design</i>)

PASTA adopte une démarche en plusieurs étapes qui part des objectifs métier et des exigences de conformité pour remonter progressivement vers les vulnérabilités techniques, ce qui la rend particulièrement adaptée lorsque la priorisation des efforts de sécurité doit être justifiée auprès de décideurs non techniques. **LINDDUN**, elle, applique une logique similaire à STRIDE mais tournée vers les atteintes à la vie privée plutôt que vers les propriétés de sécurité classiques — elle sera reprise en détail au chapitre 6.

8. Quand et comment mener un threat modeling

Le threat modeling produit le plus de valeur lorsqu'il est mené selon quelques principes pratiques éprouvés :

- **Au bon moment** : idéalement en phase de conception, avant l'écriture du code, quand les modifications restent peu coûteuses (chapitre 1). Il doit ensuite être **révisé** à chaque évolution architecturale significative — l'ajout d'un nouveau composant, l'ouverture d'une nouvelle intégration externe, ou un changement de modèle de déploiement justifient chacun une révision, même partielle.
- **En atelier collaboratif** : le threat modeling gagne à associer développeurs, architectes et personnel sécurité dans une session de travail commune, plutôt que d'être l'exercice isolé d'un seul expert sécurité déconnecté de l'équipe de développement. Les développeurs connaissent le fonctionnement réel du système bien mieux que quiconque, et leur participation directe augmente à la fois la qualité de l'analyse et l'appropriation des contre-mesures qui en découlent.
- **Documenté et priorisé** : les menaces identifiées et les contre-mesures associées doivent être consignées et priorisées, pour alimenter directement le backlog de développement — chaque contre-mesure devient une user story ou une tâche technique traçable, plutôt qu'une recommandation qui reste sans suite dans un rapport jamais relu.

À retenir

- Le threat modeling identifie systématiquement les menaces à partir d'une représentation du système (diagramme de flux de données) et de ses frontières de confiance, avant l'implémentation, en suivant les quatre questions de Shostack.
- STRIDE structure l'identification des menaces par catégorie (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege), en miroir des propriétés de sécurité visées ; DREAD ou une matrice de risque classique aident ensuite à prioriser.
- Les arbres d'attaque offrent une vue hiérarchique des chemins d'attaque possibles vers un objectif, utile pour identifier des contre-mesures à fort effet de levier.
- Les méthodologies alternatives (PASTA, LINDDUN) offrent des perspectives complémentaires selon le contexte (risque métier, vie privée).
- Un threat modeling efficace est mené en amont, collaborativement, et débouche sur des contre-mesures documentées et intégrées au backlog — ce n'est pas un exercice isolé et sans suite.

Chapitre 3 — Principes de conception sécurisée

Security by design

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Moindre privilège (least privilege)

Chaque composant, utilisateur ou processus ne doit disposer que des droits strictement nécessaires à l'accomplissement de sa tâche, ni plus. Ce principe, parmi les plus anciens et les plus universellement reconnus de la sécurité informatique, s'applique aussi bien aux comptes utilisateurs qu'aux services techniques et aux processus automatisés.

Application concrète : un service applicatif se connectant à une base de données ne doit pas utiliser un compte disposant de droits d'administration complets (création de tables, suppression de bases, gestion des utilisateurs) si seules des opérations de lecture et d'écriture sur des tables spécifiques sont nécessaires à son fonctionnement. De même, un pipeline d'intégration continue qui a seulement besoin de lire un dépôt de code ne doit pas disposer d'un jeton d'accès permettant également d'en modifier les paramètres ou d'en supprimer les branches.

L'intérêt de ce principe est direct : il limite l'impact d'une compromission. Un composant compromis — que ce soit par une faille applicative, un identifiant volé ou une dépendance malveillante — ne peut nuire qu'à hauteur de ses propres privilèges. Un service de lecture seule compromis expose au pire une fuite de données ; le même service doté de droits d'administration complets, une fois compromis, peut permettre à un attaquant de détruire ou de modifier l'ensemble du système.

2. Défense en profondeur (defense in depth)

La défense en profondeur consiste à superposer plusieurs couches de contrôles indépendantes, de sorte que la défaillance d'une seule couche ne compromette pas l'ensemble du système. L'origine militaire de l'expression est révélatrice : il s'agit de ralentir et de limiter la progression d'un assaillant à travers des lignes de défense successives, plutôt que de miser sur une unique ligne infranchissable.

Exemple typique pour une application web exposée sur Internet :

Couche	Rôle
Pare-feu réseau	Filtrer le trafic non légitime avant qu'il n'atteigne l'infrastructure applicative
Pare-feu applicatif (WAF)	Détecter et bloquer des motifs d'attaque connus au niveau des requêtes HTTP
Validation des entrées côté serveur	Rejeter les données malformées ou inattendues, indépendamment de tout contrôle côté client contournable
Requêtes préparées	Empêcher structurellement l'injection SQL, quelle que soit la donnée fournie en entrée
Chiffrement des données sensibles au repos	Limiter l'impact d'un accès non autorisé direct au support de stockage
Journalisation et détection d'anomalies	Détecter une compromission qui aurait franchi les couches précédentes

Le point essentiel de ce principe est qu'**aucune couche seule n'est infaillible** : un pare-feu applicatif peut être contourné par une nouvelle variante d'attaque non encore reconnue, une validation d'entrée peut comporter un oubli. C'est la combinaison de plusieurs couches indépendantes, chacune couvrant des angles différents, qui réduit fortement la probabilité qu'une seule faille suffise à compromettre le système entier.

3. Échec sécurisé (fail-safe / fail-secure)

En cas d'erreur ou de défaillance technique, un système doit basculer vers l'état le plus sûr, jamais vers un état permissif par défaut. Ce principe, souvent négligé dans le code applicatif courant, distingue une gestion d'erreurs correcte d'une gestion d'erreurs dangereuse.

Exemple concret : si un mécanisme de vérification d'autorisation échoue techniquement — exception non gérée, dépassement de délai (timeout) lors de l'appel à un service d'autorisation externe, erreur de connexion à une base de permissions — le comportement par défaut du système doit être de **refuser** l'accès, jamais de l'accorder par défaut faute de réponse claire.

Une erreur de logique qui inverse ce principe constitue une classe de bug de sécurité récurrente et particulièrement dangereuse, car elle passe souvent inaperçue en test tant que le chemin d'erreur n'est pas explicitement déclenché :

```
# Dangereux : en cas d'exception, l'accès est accordé par défaut
def verifier_acces(utilisateur, ressource):
    try:
        return service_autorisation.est_autorise(utilisateur, ressource)
    except Exception:
        return True # échec permissif : à proscrire

# Correct : en cas d'exception, l'accès est refusé par défaut
def verifier_acces(utilisateur, ressource):
    try:
        return service_autorisation.est_autorise(utilisateur, ressource)
    except Exception:
        return False # échec sécurisé
```

Ce principe s'applique au-delà du seul code applicatif : un système de contrôle d'accès physique doit, en cas de coupure de courant, verrouiller les portes plutôt que les déverrouiller (sauf considérations de sécurité incendie qui priment alors sur la sécurité informatique), et un mécanisme de validation de certificat doit rejeter une connexion en cas d'échec de vérification, plutôt que de l'accepter silencieusement.

4. Médiation complète (complete mediation)

Chaque accès à une ressource protégée doit être vérifié systématiquement, à chaque tentative, sans mise en cache non maîtrisée d'une autorisation antérieure qui pourrait être devenue caduque entre-temps. Ce principe s'oppose à une pratique répandue mais dangereuse : vérifier les droits d'un utilisateur une seule fois « à l'entrée » d'une session, puis considérer cette vérification valide pour toute la durée de la session sans nouvelle vérification.

Le problème concret est le suivant : si un droit est révoqué en cours de session (un employé quitte l'organisation, un rôle est retiré à un utilisateur, une ressource change de niveau de sensibilité), une vérification effectuée uniquement à la connexion initiale ne reflète plus la réalité des droits actuels. Un utilisateur dont l'accès vient d'être révoqué peut ainsi continuer à agir sur le système tant que sa session active n'expire pas, alors même que son autorisation formelle a déjà été retirée.

La médiation complète implique donc de vérifier l'autorisation à **chaque** opération sensible, et pas seulement au moment de l'authentification initiale. Cela ne signifie pas nécessairement une requête réseau vers un service d'autorisation à chaque appel — un cache d'autorisation à durée de vie courte et explicitement invalidée lors d'un changement de droit peut constituer un compromis acceptable — mais la conception doit traiter cette question de façon délibérée, plutôt que de supposer implicitement qu'une vérification initiale suffit pour toute la durée d'une session.

5. Séparation des tâches (separation of duties)

Ce principe consiste à répartir des opérations sensibles entre plusieurs personnes ou composants distincts, de sorte qu'aucune entité unique ne puisse, seule, réaliser une action critique de bout en bout sans contrôle croisé indépendant. L'objectif n'est pas seulement de limiter les conséquences d'une compromission technique, mais aussi de se prémunir contre une action malveillante interne, dont la probabilité augmente avec l'ampleur des privilèges concentrés sur une seule entité.

Exemple organisationnel classique : la personne qui valide un paiement ne doit pas être la même que celle qui l'initie, afin qu'une fraude nécessite la collusion de deux personnes plutôt que l'action isolée d'une seule. Exemple technique directement transposable au développement logiciel : séparer les clés de signature de code de production des systèmes de build automatisés accessibles à un grand nombre de développeurs, de sorte qu'une compromission du système de build ne permette pas à elle seule de signer et publier un artefact malveillant sous une identité de confiance.

Ce principe se retrouve également dans la gestion des accès à l'infrastructure critique : l'équipe qui développe une application ne devrait pas nécessairement disposer, seule et sans validation, des droits pour modifier directement les données de production, ce qui introduit un contrôle croisé naturel avant toute intervention sensible.

6. Simplicité de conception (economy of mechanism)

Un système simple est plus facile à analyser, à auditer et, dans une certaine mesure, à prouver correct qu'un système complexe. Ce principe part d'un constat empirique bien établi en génie logiciel : la complexité est l'ennemie de la sécurité, car elle multiplie les cas limites, les interactions imprévues entre composants, et donc la surface d'attaque effective d'un système — même lorsque cette surface n'est pas immédiatement visible.

Un mécanisme de sécurité complexe est également plus difficile à configurer correctement par les équipes qui l'exploitent au quotidien : une politique de contrôle d'accès à dizaines de règles imbriquées est nettement plus sujette aux erreurs de configuration qu'une politique simple et lisible, même si la première paraît a priori plus « puissante » sur le papier.

Ce principe justifie de préférer des architectures et des dépendances minimales à des solutions sur-conçues « au cas où », intégrant des fonctionnalités ou des flexibilités qui ne répondent à aucun besoin réel identifié, mais qui augmentent mécaniquement la surface à sécuriser, tester et maintenir dans la durée.

7. Conception ouverte (open design) — rappel du principe de Kerckhoffs

Ce principe reprend directement un acquis du module *Cryptographie* : la sécurité d'un système ne doit jamais reposer sur le secret de sa conception (*security through obscurity*), mais sur la robustesse de mécanismes qui restent sûrs même connus dans le détail par un attaquant. Seuls les secrets d'authentification proprement dits — clés cryptographiques, mots de passe — doivent rester confidentiels ; le design du système lui-même peut, en principe, être examiné publiquement sans que cela compromette sa sécurité.

Ce principe a une conséquence pratique importante pour la conception : un système dont la sécurité s'effondrerait si son code source ou son architecture devenaient publics n'était, en réalité, jamais sécurisé — il reposait sur une illusion de protection par la confidentialité de sa conception, une hypothèse fragile qui ne résiste ni à une fuite de code source ni à de la rétro-ingénierie.

8. Minimisation de la surface d'attaque

Réduire le nombre de points d'entrée exposés — fonctionnalités actives, ports réseau ouverts, points d'accès d'API, dépendances tierces intégrées — au strict nécessaire au fonctionnement du système. Ce principe rejoint directement la simplicité de conception évoquée plus haut, mais mérite d'être traité séparément tant son application concrète est spécifique.

Chaque fonctionnalité, chaque point d'entrée d'API ou chaque dépendance supplémentaire constitue une surface d'attaque potentielle additionnelle : du code qui n'est pas exécuté ne peut, par définition, pas contenir de vulnérabilité exploitable en pratique, tandis que chaque ligne de code active, chaque port ouvert et chaque bibliothèque tierce intégrée doit être sécurisée et maintenue à jour dans la durée. Un serveur qui expose par défaut des services de gestion à distance non utilisés, ou une application qui embarque une bibliothèque volumineuse pour n'en utiliser qu'une fonction marginale, augmentent inutilement leur exposition sans bénéfice fonctionnel correspondant.

9. Valeurs par défaut sécurisées (secure by default)

Un système livré ou installé doit être sécurisé « dès la sortie de la boîte », sans que l'utilisateur ou l'administrateur qui le déploie ait à activer explicitement les protections de base pour en bénéficier. Concrètement, cela signifie par exemple que le chiffrement des communications doit être activé par défaut plutôt qu'optionnel, que les comptes par défaut fournis avec un logiciel doivent être désactivés ou contraints à un changement de mot de passe immédiat, et que les ports réseau non essentiels au fonctionnement de base doivent rester fermés jusqu'à ce qu'un besoin explicite justifie leur ouverture.

Ce principe rejoint directement l'observation du chapitre 1 : la sécurité facile et activée par défaut protège en pratique bien plus efficacement que la sécurité techniquement supérieure mais laissée à l'initiative d'un utilisateur qui, dans les faits, ne l'activera souvent jamais — que ce soit par méconnaissance, par manque de temps, ou simplement parce que l'option reste invisible dans une interface de configuration.

À retenir

- Ces principes (Saltzer et Schroeder, et compléments ultérieurs) forment un socle de conception réutilisable indépendamment de la technologie employée — d'un système d'exploitation historique à une architecture Cloud moderne.
- Ils se combinent entre eux : la défense en profondeur suppose l'application cohérente du moindre privilège à chaque couche ; l'échec sécurisé suppose une médiation complète pour être réellement efficace à chaque accès.
- Simplicité de conception et minimisation de la surface d'attaque poursuivent un même objectif — réduire ce qu'il y a à sécuriser — sous deux angles complémentaires (complexité interne, points d'entrée exposés).
- Aucun principe pris isolément ne suffit ; c'est leur application systématique et cohérente à l'échelle d'un système entier qui constitue une véritable démarche *security by design*.

Chapitre 4 — Architecture sécurisée : segmentation et zero trust

Security by design

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Le modèle périmétrique traditionnel et ses limites

Le modèle de sécurité dit « château fort » repose sur un périmètre — typiquement matérialisé par un ou plusieurs pare-feu — séparant un réseau interne considéré de confiance d'un extérieur considéré hostile. Ce modèle a longtemps été l'approche dominante de la sécurité réseau : concentrer l'essentiel des contrôles à la frontière, et accorder une confiance largement implicite à tout ce qui se trouve à l'intérieur de cette frontière une fois franchie légitimement.

Ce modèle s'avère aujourd'hui fragile face à des évolutions structurelles qui érodent progressivement la notion même de « périmètre » clairement délimité :

- Le **télétravail massif** implique que des utilisateurs légitimes se connectent quotidiennement depuis des réseaux extérieurs à l'organisation, rendant la notion de « réseau interne de confiance » de moins en moins pertinente comme critère de confiance à elle seule.
- L'usage généralisé du **Cloud** place données et applications hors du périmètre physique traditionnel de l'organisation, chez des fournisseurs tiers dont l'infrastructure ne relève pas d'un contrôle direct.
- Une **compromission initiale par hameçonnage** d'un poste de travail interne donne d'emblée à l'attaquant la position privilégiée d'un utilisateur « de confiance », sans qu'il ait eu besoin de franchir frontalement le pare-feu périmétrique.
- Des **prestataires tiers** disposant d'accès internes légitimes (maintenance, support, intégration) constituent un point d'entrée qui contourne, par construction, la logique périmétrique classique.

2. Segmentation réseau

Avant même l'émergence du concept de zero trust, la **segmentation réseau** constitue déjà une pratique de réduction de risque bien établie : diviser le réseau en zones distinctes, séparées par un filtrage strict entre elles, plutôt que de traiter l'ensemble du réseau interne comme une zone de confiance homogène (rappel du module *Audit organisation et technique*, chapitre 4).

Une architecture segmentée typique distingue par exemple une **DMZ** (zone démilitarisée) pour les services exposés à l'extérieur, un réseau interne lui-même cloisonné par fonction ou par niveau de sensibilité (réseau des postes utilisateurs, réseau des serveurs applicatifs, réseau des systèmes financiers), et un réseau d'administration isolé du réseau utilisateur courant, réservé aux opérations de gestion des systèmes.

L'intérêt direct de cette segmentation est de limiter la **propagation latérale** (*lateral movement*) d'un attaquant ayant compromis un premier point d'entrée : un poste utilisateur compromis dans un réseau bien segmenté ne donne pas automatiquement accès aux serveurs de base de données critiques, car un filtrage strict s'interpose entre les deux zones, forçant l'attaquant à franchir une barrière supplémentaire — et potentiellement à être détecté dans le processus.

3. Principes du modèle Zero Trust

Le modèle **zero trust**, formalisé notamment par le NIST dans sa publication SP 800-207, part d'un principe simple mais qui rompt radicalement avec la logique périmétrique : **aucune confiance implicite ne doit être accordée du simple fait qu'une requête provient du réseau interne**. Chaque accès doit être authentifié, autorisé et vérifié en continu, indépendamment de sa localisation réseau d'origine — qu'il provienne d'un poste connecté physiquement au siège de l'organisation ou d'un appareil distant sur Internet.

Principes clés

- **Vérification explicite** : authentifier et autoriser systématiquement chaque accès, en s'appuyant sur l'ensemble des données disponibles pour évaluer sa légitimité — identité de l'utilisateur, appareil utilisé et sa posture de sécurité (mis à jour, conforme aux politiques), localisation géographique, et comportement observé. Aucun de ces signaux pris isolément ne suffit ; c'est leur combinaison qui permet une décision d'accès robuste.
- **Moindre privilège appliqué dynamiquement** : accorder des accès **juste-à-temps** (*just-in-time*) et **juste-suffisants** (*just-enough-access*), c'est-à-dire des droits accordés pour la durée strictement nécessaire à une tâche donnée, plutôt que des droits larges accordés une fois pour toutes et rarement révisés par la suite.
- **Présomption de compromission** (*assume breach*) : concevoir le système comme si un attaquant était déjà présent quelque part dans l'environnement, ce qui conduit à minimiser le rayon d'action possible d'une compromission (via la segmentation fine et la micro-segmentation) tout en maximisant les capacités de détection d'une activité anormale.

4. Composants typiques d'une architecture zero trust

La mise en œuvre d'une architecture zero trust combine plusieurs briques technologiques complémentaires, chacune couvrant un aspect différent de la vérification continue :

Composant	Rôle
Gestion des identités (IAM)	Authentification forte (MFA), gestion centralisée des identités et des droits associés à chaque utilisateur ou service
Politique d'accès contextuelle	Décision d'accès fondée sur de multiples signaux (identité, posture de l'appareil, localisation, sensibilité de la ressource demandée), et non plus sur la seule appartenance au réseau interne
Micro-segmentation	Contrôle fin des flux réseau, y compris entre charges de travail internes elles-mêmes, et pas seulement à la frontière externe du réseau
Chiffrement systématique	Chiffrement de l'ensemble des flux, y compris en interne, sans supposer qu'un réseau interne serait intrinsèquement sûr et donc dispensé de chiffrement
Supervision continue	Journalisation et détection comportementale permettant de révoquer un accès en cours de session dès qu'un comportement anormal est détecté, plutôt que de se fier indéfiniment à une autorisation accordée en début de session

Ces composants ne fonctionnent pleinement que combinés : une gestion des identités robuste sans micro-segmentation laisse un attaquant ayant obtenu un accès légitime se déplacer librement une fois entré ; une micro-segmentation sans supervision continue laisse passer des comportements anormaux qui restent formellement autorisés.

5. Zero trust n'est pas un produit unique

Il est important de dissiper une confusion fréquente : zero trust est une **architecture et une philosophie de conception**, pas un produit unique que l'on pourrait acheter sur étagère et installer pour « obtenir » le zero trust. Sa mise en œuvre combine plusieurs briques technologiques existantes — IAM, micro-segmentation, chiffrement, supervision — orchestrées selon les principes présentés à la section 3.

Cette mise en œuvre s'accompagne nécessairement d'une **transformation organisationnelle**, et pas uniquement technique : elle implique une revue des processus d'accès existants (qui a accès à quoi, et pour combien de temps), et une évolution de la culture des équipes vers une logique de vérification continue plutôt que de confiance acquise une fois pour toutes. Un fournisseur qui promet un produit « zero trust clé en main » sans que l'organisation revoie ses propres processus d'accès n'apporte, au mieux, qu'une partie de la solution.

6. Migration progressive

Une migration complète vers zero trust est rarement — et prudemment rarement — un projet mené en une seule fois (« big bang ») : une organisation qui basculerait brutalement l'ensemble de son infrastructure existante prendrait un risque opérationnel disproportionné par rapport au bénéfice de sécurité immédiat.

Elle se conduit généralement de façon **incrémentale**, en priorisant en premier lieu les actifs les plus critiques, identifiés au préalable par la démarche de gestion des risques du module *Gouvernance*. Cette priorisation permet de concentrer l'effort initial là où le gain de sécurité est le plus significatif, plutôt que de disperser les ressources sur l'ensemble du système d'un coup.

Durant cette transition, des **architectures hybrides** combinent nécessairement des éléments périmétriques hérités de l'existant avec des contrôles zero trust nouvellement introduits, le temps que la migration progresse. Cette coexistence transitoire n'est pas un défaut de conception mais une réalité opérationnelle incontournable pour toute organisation disposant déjà d'une infrastructure établie de longue date.

À retenir

- Le modèle périmétrique traditionnel suppose une confiance implicite au réseau interne, une hypothèse de moins en moins tenable face au télétravail, au Cloud et aux menaces internes ou latérales.
- La segmentation réseau, préalable historique au zero trust, limite déjà la propagation latérale d'une compromission en cloisonnant le réseau en zones distinctes.
- Le zero trust remplace la confiance implicite par une vérification systématique et continue de chaque accès (vérification explicite, moindre privilège dynamique, présomption de compromission), quelle que soit sa provenance.
- Sa mise en œuvre combine des briques technologiques existantes (IAM, micro-segmentation, chiffrement, supervision) et s'accompagne d'une transformation organisationnelle, pas seulement technique.
- Une migration réaliste se conduit de façon progressive et priorisée, via des architectures hybrides transitoires, plutôt qu'en un seul projet global.

Chapitre 5 — Cycle de développement sécurisé (DevSecOps)

Security by design

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Le cycle de développement sécurisé (Secure SDLC)

Un **Secure SDLC** (*Software Development Life Cycle* sécurisé) consiste à intégrer des activités de sécurité à chaque phase du cycle de vie du développement logiciel, plutôt que de traiter la sécurité comme une étape de validation finale isolée, menée juste avant la mise en production par une équipe séparée du reste du projet. Cette approche transversale est l'application directe, au niveau du processus de développement lui-même, du principe de *security by design* introduit au chapitre 1.

Phase	Activités de sécurité
Exigences	Exigences de sécurité explicites formulées dès le cadrage, classification de la sensibilité des données traitées par le système
Conception	Threat modeling (chapitre 2), revue d'architecture par des personnes compétentes en sécurité
Développement	Bonnes pratiques de codage sécurisé, revue de code par les pairs, analyse statique du code (SAST)
Tests	Tests de sécurité automatisés dérivés du threat modeling, analyse dynamique (DAST), tests d'intrusion ponctuels sur les composants critiques
Déploiement	Durcissement de la configuration des environnements, gestion sécurisée des secrets d'authentification
Exploitation	Supervision continue, gestion des correctifs de sécurité, procédure de réponse à incident

Chaque phase produit ainsi ses propres artefacts de sécurité, qui s'enchaînent logiquement : le threat modeling de la phase de conception alimente directement les cas de test de sécurité de la phase de tests, qui eux-mêmes s'appuient sur les exigences formulées en amont.

2. « Shift left » : déplacer la sécurité en amont

Le principe du **shift left** consiste à déplacer les activités de sécurité le plus tôt possible dans le cycle de développement — littéralement « vers la gauche » d'une frise chronologique de projet, où les phases amont (exigences, conception) figurent à gauche et les phases aval (déploiement, exploitation) à droite.

Ce principe découle directement de l'observation détaillée au chapitre 1 : plus une faille de sécurité est détectée tôt dans le cycle de développement, moins elle coûte à corriger, à la fois en effort technique et en risque opérationnel. Un test de sécurité automatisé exécuté à chaque *commit*, par exemple, détecte une régression de sécurité en quelques minutes, alors que la même régression découverte lors d'un audit externe annuel peut être restée exploitable en production pendant des mois.

3. DevSecOps : intégrer la sécurité dans DevOps

DevOps vise à accélérer et fiabiliser le cycle développement-déploiement grâce à l'automatisation — notamment via l'intégration continue et le déploiement continu (CI/CD) — et une collaboration étroite entre équipes de développement et d'exploitation, historiquement souvent cloisonnées. **DevSecOps** étend cette approche en intégrant la sécurité comme une responsabilité **partagée et automatisée** tout au long du pipeline, plutôt que comme une porte de validation manuelle et tardive gérée par une équipe sécurité séparée, intervenant en fin de parcours et perçue comme un frein au rythme de livraison.

Contrôles de sécurité automatisables dans un pipeline CI/CD

Étape du pipeline	Contrôle de sécurité	Ce qu'il détecte
Commit / pre-commit	Détection de secrets codés en dur (clés d'API, mots de passe, jetons d'accès)	Empêche qu'un identifiant sensible entre dans l'historique du dépôt avant même le premier commit
Build	Analyse statique du code (SAST)	Motifs de code dangereux détectables sans exécution (injection potentielle, gestion d'erreurs non sécurisée)
Dépendances	Analyse de composition logicielle (SCA)	Vulnérabilités déjà connues et publiquement documentées dans les bibliothèques tierces utilisées
Tests	Tests de sécurité automatisés	Cas de test dérivés directement du threat modeling mené en conception
Déploiement en environnement de test	Analyse dynamique (DAST)	Vulnérabilités observables uniquement à l'exécution, contre l'application réellement en fonctionnement
Image de conteneur	Analyse de vulnérabilités de l'image	Failles connues dans le système de base et les paquets inclus, avant publication de l'image
Infrastructure as Code	Analyse de configuration	Règles de pare-feu trop permissives, stockage mal configuré (par exemple accessible publiquement par erreur), avant même le provisionnement effectif

L'intérêt de cette automatisation est double : elle rend les contrôles de sécurité **systématiques** (ils s'exécutent à chaque changement, sans dépendre de la disponibilité ou de la mémoire d'une personne) et elle les rend **rapides**, produisant un retour d'information en quelques minutes plutôt qu'après plusieurs semaines d'attente d'un audit manuel.

4. Gestion des secrets

Les identifiants de connexion, clés d'API, jetons d'accès et certificats ne doivent **jamais** être codés en dur dans le code source, ni versionnés dans un dépôt Git — même privé. Un dépôt privé peut devenir public par erreur de configuration, et de toute façon son historique complet reste accessible à quiconque y a eu un accès légitime à un moment donné, y compris après une révocation d'accès si l'historique n'a pas été purgé.

Un exemple de ce qu'il ne faut **jamais** écrire dans un fichier de configuration versionné :

```
# À NE JAMAIS FAIRE : secret codé en dur dans le code source
API_KEY = "<CLE_SECRETE_EN_CLAIR_A_NE_JAMAIS_VERSIONNER>"
DATABASE_PASSWORD = "VOTRE_MOT_DE_PASSE_ICI"
```

La bonne pratique consiste à injecter ces valeurs au moment du déploiement, sans qu'elles transitent jamais par le dépôt de code :

```
import os

# Correct : le secret est lu depuis l'environnement d'exécution,
# jamais stocké en clair dans le code source versionné
api_key = os.environ["API_KEY"]
```

Plusieurs bonnes pratiques complémentaires renforcent cette gestion :

- L'usage de **gestionnaires de secrets dédiés** (Vault et équivalents), qui centralisent le stockage chiffré des secrets et en contrôlent finement l'accès et la journalisation.
- L'injection de **variables d'environnement** au moment du déploiement, plutôt que leur inclusion statique dans une image ou un artefact de build.
- La **rotation régulière** des secrets, qui limite la fenêtre d'exploitation d'un secret qui aurait malgré tout fuité sans que cela soit détecté immédiatement.
- Le **scan automatique** des dépôts (y compris de leur historique complet) pour détecter des secrets déjà exposés par le passé, souvent restés présents dans d'anciens commits longtemps après leur retrait apparent du code courant.

5. Gestion des vulnérabilités des dépendances tierces

La majorité du code exécuté par une application moderne ne provient pas du code écrit directement par l'équipe de développement, mais de **dépendances tierces** — bibliothèques et paquets open source réutilisés pour éviter de réimplémenter des fonctionnalités courantes. Cette réutilisation massive apporte une productivité considérable, mais introduit également une surface de risque partagée : une vulnérabilité découverte dans une dépendance largement utilisée peut affecter simultanément un très grand nombre d'applications qui l'intègrent, souvent sans que leurs équipes de développement en aient conscience tant que la dépendance n'est pas explicitement auditée.

L'**analyse de composition logicielle** (SCA, *Software Composition Analysis*) automatisée en continu permet de maintenir un inventaire à jour des dépendances utilisées et de leurs vulnérabilités connues, en s'appuyant sur des bases de données publiques de vulnérabilités régulièrement mises à jour. Couplée à une politique de mise à jour réactive — appliquer un correctif de sécurité disponible dans un délai raisonnable plutôt que de le reporter indéfiniment — cette analyse est devenue une pratique de sécurité de base incontournable pour tout projet logiciel moderne, quelle que soit sa taille.

6. Culture et organisation

DevSecOps est autant une question de **culture** que d'outillage technique : l'automatisation seule ne suffit pas si les équipes ne s'approprient pas les résultats qu'elle produit.

- La **formation des développeurs à la sécurité** est un investissement direct, souvent porté par des « *security champions* » identifiés au sein de chaque équipe de développement — des développeurs qui approfondissent leurs compétences en sécurité et servent de relais de bonnes pratiques auprès de leurs collègues, sans nécessiter la présence d'un expert sécurité dédié dans chaque équipe.
- La **responsabilité partagée** remplace un modèle où la sécurité serait reportée entièrement sur une équipe sécurité séparée intervenant en fin de cycle : chaque développeur devient acteur de la sécurité du code qu'il écrit, plutôt que simple exécutant en attente d'une validation externe.
- Le **retour d'information rapide** est un facteur déterminant d'efficacité : un développeur informé d'une faille immédiatement après son introduction — par exemple via une alerte automatisée dans son pipeline CI/CD — la corrige bien plus efficacement, avec le contexte encore frais en mémoire, qu'informé plusieurs mois plus tard lors d'un audit externe, où il doit d'abord reconstituer le contexte avant de pouvoir agir.

À retenir

- Le Secure SDLC intègre des activités de sécurité à chaque phase du développement (exigences, conception, développement, tests, déploiement, exploitation), pas uniquement en validation finale.
- Le principe du « shift left » déplace ces activités le plus tôt possible dans le cycle, cohérent avec le constat que corriger tôt coûte nettement moins cher que corriger tard.
- DevSecOps automatise ces contrôles directement dans le pipeline CI/CD (détection de secrets, SAST, SCA, tests de sécurité, DAST, scan d'image de conteneur, scan d'infrastructure as code).
- La gestion des secrets (jamais codés en dur, gestionnaires dédiés, rotation) et des dépendances tierces vulnérables (SCA continue, mise à jour réactive) sont deux enjeux opérationnels majeurs et récurrents du développement logiciel moderne.
- DevSecOps est autant une transformation culturelle (responsabilité partagée, security champions, retour d'information rapide) qu'une question d'outillage automatisé.

Chapitre 6 — Privacy by design

Security by design

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définition et origine

Le *privacy by design* (protection de la vie privée dès la conception) est un cadre formalisé par Ann Cavoukian, alors commissaire à l'information et à la protection de la vie privée de la province canadienne de l'Ontario, articulé autour de sept principes fondateurs. Son objectif est d'intégrer la protection des données personnelles comme une **caractéristique intrinsèque** d'un système, présente dès sa conception, plutôt que comme un ajout réglementaire tardif plaqué sur une architecture déjà figée pour répondre a posteriori à une exigence de conformité.

Ce cadre partage avec *security by design* (chapitre 1) la même logique fondamentale — anticiper plutôt que corriger — mais l'applique spécifiquement au traitement des données personnelles, avec ses propres enjeux : consentement, finalité de la collecte, droits des personnes concernées, et conséquences potentiellement graves d'une fuite de données identifiantes pour les individus concernés, au-delà du seul préjudice pour l'organisation elle-même.

2. Les sept principes fondateurs

1. **Proactif, non réactif ; préventif, non correctif** : anticiper les risques pour la vie privée avant qu'un incident ne survienne, plutôt que de réagir après une violation de données déjà constatée. Cela implique d'évaluer les risques dès la phase de conception d'un traitement, pas seulement après son déploiement.
2. **Protection de la vie privée par défaut** : les paramètres les plus protecteurs doivent être activés sans action requise de l'utilisateur — un utilisateur qui ne modifie aucun réglage doit bénéficier du niveau de protection le plus élevé possible, et non du plus faible faute d'avoir explicitement renforcé ses paramètres.
3. **Protection intégrée à la conception** : la protection de la vie privée doit être une composante structurelle du système — présente dans le modèle de données, l'architecture et les flux de traitement — et non une fonctionnalité ajoutée après coup sous forme de module optionnel.
4. **Fonctionnalité intégrale (« somme positive », pas somme nulle)** : rechercher des solutions où fonctionnalité du système et protection de la vie privée coexistent pleinement, plutôt que de présenter systématiquement un arbitrage où l'une ne progresserait qu'au détriment de l'autre — un faux dilemme fréquent mais évitable dans la plupart des cas avec une conception soignée.
5. **Sécurité de bout en bout** : la protection doit être assurée tout au long du cycle de vie de la donnée, de sa collecte initiale jusqu'à sa suppression définitive, en passant par son stockage, son traitement et son éventuel partage — une protection qui s'arrêterait à l'une de ces étapes laisserait une fenêtre d'exposition.
6. **Visibilité et transparence** : les pratiques réelles de traitement des données doivent être vérifiables par des tiers de confiance ou par les personnes concernées elles-mêmes, et pas seulement déclarées dans une politique de confidentialité qui ne refléterait pas fidèlement le traitement effectif.
7. **Respect de la vie privée des utilisateurs** : la conception doit rester centrée sur l'intérêt de la personne concernée, avec des interfaces et des paramètres de confidentialité clairs et facilement accessibles, plutôt que délibérément complexes ou dissuasifs pour décourager leur usage effectif.

3. Traduction juridique : article 25 du RGPD

Le **RGPD** (Règlement Général sur la Protection des Données, adopté par l'Union européenne, mais largement pris en référence dans le monde au-delà de sa portée juridique territoriale directe) reprend explicitement ce principe dans son **article 25**, intitulé « protection des données dès la conception et par défaut » (*privacy by design and by default*). Cet article transforme une bonne pratique auparavant volontaire en **obligation légale** : il impose concrètement au responsable de traitement de mettre en œuvre des mesures techniques et organisationnelles appropriées dès la conception d'un traitement de données personnelles, et non uniquement au moment de sa mise en service.

Cette exigence juridique a une conséquence directe sur les pratiques de conception logicielle : la démonstration de conformité ne peut plus se limiter à une politique de confidentialité rédigée après coup, elle doit s'appuyer sur des choix de conception effectivement documentés et traçables — ce qui rejoint directement les pratiques d'analyse d'impact décrites à la section 6 ci-dessous.

4. Principes de minimisation

La minimisation est le principe opérationnel le plus directement actionnable en conception : à chaque décision de conception impliquant des données personnelles, la question à se poser est « en avons-nous réellement besoin, sous cette forme, pour cette durée, et pour qui ? »

- **Minimisation de la collecte** : ne collecter que les données strictement nécessaires à la finalité poursuivie, jamais des données collectées « au cas où » elles seraient utiles plus tard pour un usage non encore défini au moment de la collecte. Un formulaire d'inscription qui demande une date de naissance sans qu'aucune fonctionnalité du système n'en ait besoin viole ce principe.
- **Minimisation de la conservation** : définir une durée de conservation limitée et explicitement justifiée par la finalité du traitement, avec une suppression ou une anonymisation effective au-delà de cette durée — et non une conservation indéfinie par simple facilité technique.
- **Minimisation de l'accès** : appliquer le principe du moindre privilège (chapitre 3) spécifiquement aux données personnelles, en s'assurant que seules les personnes ou les composants qui en ont réellement besoin pour une tâche donnée y ont accès.
- **Minimisation de l'identifiabilité** : privilégier, lorsque la finalité poursuivie le permet, des données pseudonymisées ou anonymisées plutôt que des données directement identifiantes — un traitement statistique agrégé n'a par exemple généralement pas besoin de connaître l'identité précise de chaque individu concerné.

5. Techniques de protection de la vie privée

Plusieurs techniques concrètes permettent de traduire les principes de minimisation en mesures effectives :

Technique	Principe	Remarque
Pseudonymisation	Remplacer un identifiant direct par un pseudonyme, la ré-identification restant possible avec une information additionnelle gardée séparément	Réduit le risque en cas de fuite partielle, mais la donnée reste juridiquement une donnée personnelle tant que la ré-identification reste possible
Anonymisation	Rendre la ré-identification impossible, y compris par recoupement avec d'autres sources de données disponibles	Plus difficile à garantir réellement en pratique que ce que l'on croit souvent, en particulier sur des jeux de données riches en attributs croisables
Chiffrement	Protéger la confidentialité des données personnelles au repos et en transit	Lien direct avec le module <i>Cryptographie</i> ; protège contre l'accès non autorisé mais ne réduit pas, à lui seul, le volume de données collecté
Confidentialité différentielle (differential privacy)	Ajout de bruit statistique calibré à des résultats d'agrégation pour empêcher la ré-identification d'un individu tout en préservant l'utilité statistique globale du jeu de données	Technique avancée, utile notamment pour la publication de statistiques agrégées sans exposer les contributions individuelles
Contrôle d'accès et journalisation	Limiter et tracer précisément qui accède à quelles données personnelles, et à quel moment	Complète la minimisation de l'accès en la rendant vérifiable a posteriori

Ces techniques ne s'excluent pas mutuellement : un système bien conçu combine généralement plusieurs d'entre elles selon la sensibilité et l'usage de chaque catégorie de données traitées.

6. Analyse d'impact relative à la protection des données (AIPD/DPIA)

Pour un traitement de données présentant un risque élevé pour les droits des personnes concernées — par exemple un traitement à grande échelle de données sensibles, ou un dispositif de surveillance systématique — une **analyse d'impact formelle** est requise par de nombreuses réglementations, dont le RGPD.

Cette analyse documente plusieurs éléments de façon structurée : la **finalité** précise du traitement envisagé, sa **nécessité** et sa **proportionnalité** au regard de cette finalité, une évaluation des **risques** pour les droits et libertés des personnes concernées, et enfin l'identification des **mesures** techniques et organisationnelles destinées à atténuer ces risques identifiés.

Cette démarche est directement apparentée à l'analyse de risque enseignée dans le module *Gouvernance et gestion des risques SI*, mais appliquée spécifiquement au domaine de la vie privée : elle en reprend la logique générale (identifier, évaluer, atténuer) tout en la centrant sur l'impact potentiel pour les personnes concernées, plutôt que sur le seul impact pour l'organisation qui traite les données.

7. LINDDUN : threat modeling orienté vie privée

LINDDUN est un complément direct au threat modeling présenté au chapitre 2 : la méthode structure l'identification de menaces spécifiques à la vie privée, sur le même principe organisationnel que STRIDE (une classification en catégories, examinées systématiquement pour chaque composant), mais orientée vers les atteintes à la vie privée plutôt que vers les propriétés de sécurité classiques.

Lettre	Menace
L	Linkability (liaison de données entre elles, permettant de relier des informations autrement disjointes à une même personne)
I	Identifiability (identifiabilité, possibilité d'identifier une personne à partir de données a priori anonymes)
N	Non-repudiation (non-répudiation, ici comme menace : impossibilité pour une personne de nier de façon plausible une action, même quand cela nuirait à sa vie privée)
D	Detectability (détectabilité de l'existence même d'une donnée ou d'un événement concernant une personne)
D	Disclosure of information (divulcation d'information personnelle)
U	Unawareness (méconnaissance par la personne concernée de la collecte ou de l'usage fait de ses données)
N	Non-compliance (non-conformité aux exigences légales et réglementaires applicables)

Comme pour STRIDE, l'usage de LINDDUN consiste à examiner systématiquement chaque composant et chaque flux d'un diagramme de traitement de données au regard de ces sept catégories, afin de ne pas se limiter aux risques de vie privée les plus évidents ou les plus médiatisés.

À retenir

- Le privacy by design intègre la protection des données personnelles comme caractéristique par défaut d'un système, via ses sept principes fondateurs (Ann Cavoukian), formalisé juridiquement par l'article 25 du RGPD et des dispositions équivalentes ailleurs.
- La minimisation (collecte, conservation, accès, identifiabilité) est le principe opérationnel le plus directement actionnable en conception, appuyée par des techniques concrètes : pseudonymisation, anonymisation, chiffrement, confidentialité différentielle.
- L'analyse d'impact (AIPD/DPIA) documente finalité, nécessité, proportionnalité et mesures d'atténuation pour les traitements à risque élevé, en miroir de l'analyse de risque du module Gouvernance.
- LINDDUN transpose la logique de threat modeling (STRIDE) spécifiquement aux risques pour la vie privée, complétant l'analyse de sécurité classique du chapitre 2.